DAY 11 | POST-READ

React and APIs: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
&&	Render something or nothing. Beware of 0: use count > 0 &&.
Ternary and early return	cond ? A : B to choose; if (loading) return ... keeps the main view simple.
Lists	items.map(i => <Item key={i.id} />). Keys must be stable and unique.
Derived data	Filter and sort while rendering; do not store computed copies in state.
useEffect	Runs after render. [] means once on mount. Use it to start fetching.
Fetch states	loading, error, empty and data. Design a screen for each.
Keys off the front end (NEW v4)	Anything in the bundle is public. VITE_ variables are inlined. Call your own API.
POST and PATCH from React	fetch with method, JSON headers and body; update state with the reply.
CORS	Needed when front end and API have different origins (ports count).

Part B: Going beyond today

1. Cleaning up effects

```
useEffect(() => {
  const controller = new AbortController();
  fetch(url, { signal: controller.signal }).then(...);
  return () => controller.abort();   // runs if the component disappears
}, [url]);
```

2. A tiny custom hook

Custom hooks package reusable logic, much like FastAPI dependencies.

```
function useFetch(url) {
  const [state, setState] = useState({ loading: true, data: null, error: null });
  useEffect(() => { ... }, [url]);
  return state;
}
```

3. Optimistic updates

Update the screen immediately when the agent closes a ticket, then call the API. If the call fails, undo the change and show an error. Apps feel much faster this way.

4. Secret scanning

Tools can scan your repository and build output for secrets on every commit. You will add secret scanning to the CI pipeline on Day 20. If a key ever leaks, revoke it first, then fix the code.

5. Backend for front end (BFF)

A small server made just for one front end. It holds secrets, combines calls to several services and returns exactly what the screen needs. Your FastAPI app plays this role for SmartDesk.

Part C: Check yourself

- Why can {count && Items } show a stray 0?
- What problem do keys solve?
- Name the four states every data screen should handle.
- Why is a VITE_ environment variable not a safe place for an API key?
- Why does a React app on port 5173 need CORS to call FastAPI on port 8000?

Part D: Practice ideas for tonight

- Add a Retry button to the error state of your ticket list.
- Run npm run build on your React app and search dist/ for anything that looks like a key.
- Add an edit mode: double click a ticket title to edit it, then PATCH it.

Coming up on Day 12: the first GenAI day: how LLMs generate text, the 2026 model landscape, tokens and cost, and your first LLM API call, made safely from the server.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today